

Forge Beyond Algebra

Anchored analytic certificates, integrating-factor search, and quantified infinite tails

Concrete extensions to the audited Forge design

Prepared for Vladimir Reshetnikov

15 September 2026

Abstract

The current Forge design already supplies a substantial collection of structural, algebraic, finite-state, and finite-summation workers. Repeating those proposals would add little. This article develops three complementary analytic workers: exact anchored remainders that preserve a function's order of vanishing; search for integrating-factor differential ladders proving inequalities on an unbounded half-line; and rational tail barriers that establish summability, explicit tail estimates, and executable Cauchy moduli. A companion harmonic minorant supplies a genuinely different negative outcome: certified divergence of eventually positive rational series.

The proposal specifies certificate languages, search algorithms, mathematical soundness proofs, limitations, library interfaces, and an integration sequence. For the rational-series fragment, an elementary positive-shift argument gives a constructive convergence/divergence classification and an explicit certificate construction, rather than a bounded search heuristic alone. Existing Taylor-model and verified-numerics work is credited and reused; these are extensions relative to the inspected Forge snapshot, not claims to have invented interval arithmetic, Taylor models, integrating factors, or convergence comparison.

The accompanying standard-library Python implementation produced 246 accepted certificate records, representing 230 distinct source/domain subjects. A separate sparse-polynomial checker replays them with search imports blocked; 738 specified mutations are rejected. Symbolic cross-checks, numerical enclosure diagnostics, and exact least-index witness checks accompany the corpus. These are component experiments, not Lean proofs or tactic benchmarks. No Lean executable was available in the authoring environment, so the Lean bridge specimens and all end-to-end integrations remain explicitly uncompiled.

Audited Forge revision:

`c98e47c5f804e92880fc1d0e378c1b95832b685c`

The archive includes editable $\text{\LaTeX}$, this PDF, executable searches, standalone replay, tests, certificates, measurements, and implementation notes.

Contents

1	The contribution, and what is deliberately not repeated	3
1.1	A repository-relative delta ledger	3
1.2	Why these are useful proof shapes	4
1.3	What the experiments establish	4
2	A worked example: do not erase the zero	4
3	Worker A: exact anchored remainder arithmetic	5
3.1	The semantic object	5
3.2	Primitive certificates	5
3.3	Composition rules	6
3.4	Extracting the order of vanishing	7
3.5	Search, cost, and a useful completeness statement	7
4	Worker B: integrating-factor differential ladders	8
4.1	A global proof rule	8
4.2	An exact finite language	9
4.3	Bounded search with a small candidate set	9
4.4	A precise incompleteness obstruction	10
4.5	What is and is not new	10
5	Worker C: rational tail barriers	10
5.1	Why a finite telescoper is not enough	10
5.2	A polynomial residual with all signs accounted for	11
5.3	Exact optimisation of the barrier constant	12
6	A constructive classification for rational series	12
6.1	Making every shifted coefficient positive	12
6.2	Convergence certificates	13
6.3	A certified divergence outcome	13
6.4	From summability to asymptotic tail constants: a next extension	14
7	Synthesising and checking Cauchy moduli	15
7.1	The least admitted index	15
7.2	Rational accuracy versus an arbitrary real accuracy	15
8	How the workers cooperate with Forge	16
8.1	Dispatch based on the shape of the obligation	16
8.2	A cross-worker example	16
8.3	Failure diagnostics that change the next attempt	17
8.4	A concrete budget policy	17

9 Libraries, version boundaries, and proof reconstruction	17
9.1 Reuse LeanCert; do not write a second interval library	17
9.2 The observed toolchain mismatch	18
9.3 Two trust choices, and one concrete import hazard	18
9.4 A concrete dependency map	18
10 The missing Lean layer, specified narrowly	19
10.1 Proposed module boundaries	19
10.2 The three theorem families to formalise	20
10.3 Source obligations that must be explicit	20
10.4 What the supplied Lean file demonstrates	20
11 Experimental evidence and its limits	21
11.1 Environment and reproduction	21
11.2 Anchored corpus and ablation	21
11.3 Ladder corpus and its planted component	22
11.4 Tail corpus, witnesses, and exact controls	22
11.5 Replay, mutations, and explicit non-successes	22
11.6 What remains unmeasured	23
12 An implementation and evaluation programme	23
12.1 Acceptance gates rather than speculative deadlines	23
12.2 The benchmark that would demonstrate practical value	24
13 Conclusion	24
A Certificate schemas and replay contract	24
B Using the archive	25
C Source audit and attribution	26

1 The contribution, and what is deliberately not repeated

The reviewed Forge repository describes a certificate-producing proof-planning layer above Lean’s **grind**, not an installed tactic. Its merged article incorporates eighteen proposal packages. The extension round already includes observable-space closure, polynomial-ideal closure, finite-algebra covers, finite Kripke countermodels, boundary-safe telescoping, and Ore-operator transport. The earlier round already includes induction and generalisation, witness synthesis, nonlinear polynomial certificates, and Boolean/theory cooperation [1, 2, 3].

The right question is therefore not “what other solver can we name?” It is: *which important proof obligations still require a mathematical representation that the audited workers do not provide?* The proposal here addresses analytic contact, global differential comparison, and infinite convergence. The common pattern is to turn an infinite family of analytic obligations into a finite algebraic certificate, while retaining the theorem that licenses that reduction.

1.1 A repository-relative delta ledger

Already present or established	Not claimed as a contribution	Addition developed here
Polynomial cones, quadratic certificates, Bernstein subdivision, square-factor/Sturm reasoning	Another polynomial positivity engine	A source-preserving analytic remainder that exposes a <i>polynomial quotient</i> without losing the zero at the anchor.
Invariant ideals and finite-dimensional closures	Renaming invariant generation	A first-order differential comparison certificate that reconstructs positivity on all of $[0, \infty)$ from finitely many exact identities.
Finite Gosper/creative telescoping and boundary-safe sums	Another finite summation identity	A tail potential proving summability before using an infinite sum, together with a strict Cauchy modulus and a divergence alternative.
Arithmetic witness synthesis	The general idea of synthesising witnesses	An explicit least-index construction for a certified tail bound, checked by two rational inequalities.
Source-owned evidence, opaque receipts, scope discipline, and re-elaboration	Another general trust architecture	The new source obligations: analytic domain, anchor, interval orientation, natural-index conversion, eventual sequence identity, and summability.
Verified numerics and Taylor models in the wider literature	Inventing interval arithmetic or Taylor arithmetic	A Forge-specific routing and certificate layer, with concrete reuse decisions and new fragment-specific algorithms.

The audit used the repository README, the merged baseline section, the provenance ledger, the conclusion, the relevant nonlinear section, and the Leant/Djex review. It is *not*

a claim that every line of all eighteen original submissions was reread. The absence claim is correspondingly scoped: these three workers were not found as developed capabilities in the audited merged material [4, 5, 6].

1.2 Why these are useful proof shapes

An algebraic worker may regard $\sin x$ as an opaque real atom. That is safe, but it does not know that $x - \sin x$ has a cubic zero at the origin. A bounded numerical worker may prove an inequality on $[0, 1]$ while saying nothing about $[1, \infty)$. A finite telescoping identity can be valid even when an infinite series diverges. Each gap requires a different mathematical bridge, not a larger search budget.

The proposed output is stronger than a yes/no heuristic. An anchored worker can return $f(x) \geq \delta x^m$ on an exact interval. A ladder worker can return a chain of differential identities proving a global sign. A tail worker can return an explicit N , a bound on every remaining finite partial sum, summability, and an executable modulus. Its divergence outcome carries a harmonic comparison, not merely an exhausted convergence search.

1.3 What the experiments establish

The implemented searches and replay checker use exact rational arithmetic. The search polynomial representation is dense; the checker has a separate sparse implementation and imports no search module. The corpus has 62 anchored records, 43 ladder records, 81 barrier records, and 60 divergence records. Two elementary examples are repeated across named subfamilies, and seven series are optimised at three different cutoffs; consequently there are 230 distinct source/domain subjects, not 246 distinct theorems. Section 11 makes the units and these overlaps explicit.

No result here establishes a speedup over `grind`, `Mathlib tactics`, or `LeanCert`. Small examples may already have a library lemma or an easy manual proof. Their purpose is to exhibit the certificate and the search, not to make an existing tactic look weak. Nor does another collection of Python experiments close Forge’s existing Lean integration gap. The repository’s own conclusion correctly distinguishes algorithm design from shipping a checked end-to-end pipeline [3].

2 A worked example: do not erase the zero

Consider proving $x - \sin x \geq 0$ for $0 \leq x \leq 1/2$. A fourth-order model is

$$\sin x = x - \frac{x^3}{6} + x^4 r(x), \quad |r(x)| \leq \frac{5}{108}.$$

The rational remainder bound follows from the majorant in Section 3.2. Subtracting gives

$$x - \sin x = x^3 \left(\frac{1}{6} - x r(x) \right).$$

Since $x \geq 0$ and $x \leq 1/2$,

$$\frac{1}{6} - x r(x) \geq \frac{1}{6} - \frac{1}{2} \frac{5}{108} = \frac{31}{216} > 0.$$

Thus the same certificate proves the stronger fact

$$x - \sin x \geq \frac{31}{216} x^3 \quad (0 \leq x \leq 1/2). \tag{1}$$

In particular it also proves strict positivity whenever $x > 0$.

The difference between this and an unstructured interval calculation is exact. If we replace $x^4 r(x)$ by a uniform error interval *before* factoring, the lower bound becomes

$$0 - \left(\frac{1}{2}\right)^4 \frac{5}{108} = -\frac{5}{1728} < 0.$$

No contradiction has occurred: the lower bound is valid but too weak. It has forgotten that both the main term and the error vanish at zero at different rates. Preserving that information changes what can be certified without excluding the endpoint.

There is no division by x in the certificate theorem. The factorisation is an identity, and $x = 0$ is covered directly. This is important in Lean: replacing $f(x)$ by $f(x)/x^3$ and forgetting a nonzero side condition would prove a different statement. The representation below avoids that trap structurally.

This example is a demonstration of the worker, not a claim that the basic inequality for sine is missing from Mathlib. The general utility appears when several analytic subexpressions cancel to high order and the appropriate quotient is not a named theorem.

3 Worker A: exact anchored remainder arithmetic

Evidence status. Implemented in `jets.py`; independently replayed in Python. The semantic theorem is proved below on paper, not in Lean.

3.1 The semantic object

Definition 3.1 (Anchored enclosure). Fix $b > 0$ and $K \geq 1$. An anchored enclosure of $f : [0, b] \rightarrow \mathbb{R}$ is a pair (p, I) , where $p \in \mathbb{Q}[x]$ has degree less than K and $I = [\ell, u]$ is a rational interval, such that

$$\forall x \in [0, b], \quad \exists r \in [\ell, u], \quad f(x) = p(x) + x^K r. \quad (2)$$

The existential formulation deliberately does not require dividing by x^K . It also does not require a globally selected remainder function. At zero, only the exact constant coefficient matters. Away from zero, ordinary analytic remainder formulas supply the witness.

This is a standard Taylor-remainder representation, not a new numerical abstraction. Streeter and Dillon explicitly study intervals multiplying $(x - x_0)^K$ and compositional Taylor-mode bounds. Earlier verified interval work also combines elementary functions with Taylor estimates [9, 10]. Our contribution is its certificate use and goal-directed order-of-vanishing extraction in Forge.

Let $H_b(p)$ denote ordinary rational Horner interval evaluation on $[0, b]$. The essential property is

$$p([0, b]) \subseteq H_b(p).$$

All interval endpoints in the prototype are exact fractions. Floating-point arithmetic does not participate in search acceptance or replay.

3.2 Primitive certificates

The prototype supports rational polynomials and the primitives

$$e^{ax}, \quad \sin(ax), \quad \cos(ax), \quad \log(1 + ax), \quad \arctan(ax), \quad a \in \mathbb{Q},$$

combined by addition, negation, and multiplication. A node labelled `log1p` has the exact meaning $\log(1 + ax)$; it is not an arbitrary logarithm node whose domain is guessed later.

For exponential, sine, and cosine, use the ordinary degree- $(K - 1)$ Taylor polynomial and the symmetric remainder radius

$$R_K(a, b) = \frac{|a|^K}{K!(1 - |a|b/(K + 1))}, \quad \frac{|a|b}{K + 1} < 1. \quad (3)$$

For logarithm and arctangent, use

$$S_K(a, b) = \frac{|a|^K}{K(1 - |a|b)}, \quad |a|b < 1. \quad (4)$$

The latter guard also ensures $1 + ax > 0$ for the logarithm. The radius for arctangent is deliberately conservative: it ignores parity and alternating signs in exchange for a tiny shared checking rule.

Lemma 3.2 (Primitive enclosures). *With the corresponding guard, the indicated Taylor polynomial and interval $[-R_K, R_K]$ or $[-S_K, S_K]$ satisfy (2).*

Proof. For the exponential tail and $0 < x \leq b$, division by x^K inside this mathematical proof gives the absolute majorant

$$\sum_{j=K}^{\infty} \frac{|a|^j x^{j-K}}{j!}.$$

Its first term is $|a|^K/K!$. Each successive ratio is at most $|a|b/(K + 1) < 1$, giving (3). The sine and cosine tails are subseries of the same absolute majorant, with signs discarded.

For logarithm or arctangent, the absolute coefficient in degree $j \geq K$ is at most $|a|^j/j \leq |a|^j/K$. Summing the resulting geometric majorant after factoring x^K gives (4). The power series represent the stated functions under the guards. At $x = 0$ the truncated polynomial has the correct value, so choosing any remainder in the nonempty interval completes the proof without division at zero. $\square$

A production library should replace these symmetric bounds by stronger signed or minimax bounds where available. Nothing in the certificate interface requires this particular approximation. The current implementation makes an intentionally small, auditable choice; it does not compete with a mature numerical library's full catalogue of primitive bounds.

3.3 Composition rules

Addition and negation act componentwise. For a polynomial input s , split $s = p + x^K h$ and use $H_b(h)$ as the remainder interval. The product rule is the only nontrivial compositional case.

Suppose $f = p + x^K r$ and $g = q + x^K s$, with $r \in I$ and $s \in J$. Write

$$pq = t + x^K h, \quad \deg t < K.$$

Then

$$fg = t + x^K (h + ps + qr + x^K rs). \quad (5)$$

Consequently a valid product remainder interval is

$$H_b(h) + H_b(p)J + H_b(q)I + [0, b^K]IJ. \quad (6)$$

All operations in this expression are interval operations. They preserve containment even when occurrences are correlated; the cost of forgetting a correlation is loss of precision, not unsoundness.

Theorem 3.3 (Compositional soundness). *For every accepted expression in the prototype grammar, the enclosure computed by these rules satisfies (2).*

Proof. Induct on the expression tree. Polynomial nodes follow from exact coefficient splitting and Horner containment. Primitive nodes follow from the preceding lemma. Addition and negation follow by combining the existential remainder witnesses at the same x . For multiplication use (5) and containment in each of the four intervals in (6). $\square$

3.4 Extracting the order of vanishing

Let $m < K$ be the least index of a nonzero coefficient of p . Then $p = x^m q$ and

$$f(x) = x^m(q(x) + x^{K-m}r).$$

Compute

$$\delta = \inf(H_b(q) + [0, b^{K-m}]I). \quad (7)$$

If $p = 0$, instead set $m = K$ and $\delta = \inf I$.

Theorem 3.4 (Anchored positivity certificate). *An accepted enclosure with $\delta \geq 0$ proves*

$$f(x) \geq \delta x^m \geq 0 \quad \text{for every } x \in [0, b].$$

Proof. When $p \neq 0$, (7) is a lower bound on the factor in parentheses. Multiplication by $x^m \geq 0$ preserves the inequality. When $p = 0$, the conclusion follows directly from $f(x) = x^K r$ and $r \geq \delta$. Both arguments include zero. $\square$

The certificate stores the expression, exact bound b , order K , polynomial, remainder interval, valuation m , and margin δ . The checker recomputes all of them from the externally supplied subject. Neither a claimed zero coefficient nor a claimed margin is trusted.

3.5 Search, cost, and a useful completeness statement

The implemented search tries $K = 2, 3, \dots, K_{\max}$ at the supplied b and returns the first nonnegative margin. Unsupported primitive guards are skipped. The checker admits at most 256 expression nodes, depth 32, and order 40. Dense truncated polynomial multiplication costs $O(K^2)$ rational operations per multiplication node. Exact numerator and denominator bit growth must be accounted for separately from operation counts.

```

anchored_search(expression, exact_box, order_budget):
  for K in 2 .. order_budget:
    model := build_anchored_model(expression, exact_box, K)
    if a primitive guard is unavailable: continue
    m := first nonzero polynomial coefficient, or K
    margin := lower bound after extracting x^m
    if margin >= 0:
      return candidate(model, m, margin)
  return unknown

```

For a goal asking for a neighbourhood rather than a prescribed interval, the controller may additionally search $b_0/2^j$. This wrapper is specified, not separately executed in the corpus.

Proposition 3.5 (Local positive-germ completeness, without resource caps). *For the supported analytic grammar, suppose the first nonzero Taylor coefficient of f at zero has positive value and degree m . A fair search over orders and shrinking positive dyadic radii eventually obtains an anchored positivity certificate.*

Proof. Choose a fixed order $K > m$. On sufficiently small radii all primitive guards hold and all propagated remainder intervals remain uniformly bounded. This follows inductively from the explicit primitive radii and the algebraic product rule. The Horner interval for $q = p/x^m$ tends to the positive constant $q(0)$ as $b \rightarrow 0$. Meanwhile $[0, b^{K-m}]I$ tends to $\{0\}$. Therefore the lower endpoint in (7) is eventually positive. A fair search reaches such an order/radius pair. $\square$

This theorem does not say that the fixed-radius implementation decides analytic inequalities. Nor does it decide equality of analytic functions. For example, $\sin^2 x + \cos^2 x - 1 = 0$ is true, but the prototype’s independently propagated symmetric errors do not cancel exactly; its nonnegativity search returns **unknown** under the recorded cap. The existing algebraic/rewrite layer should recognise the identity before numerical approximation is attempted.

4 Worker B: integrating-factor differential ladders

Evidence status. Exact search and Python replay are implemented in `ladders.py` and `checker.py`; derivative identities are independently cross-checked with SymPy. Lean reconstruction is specified, not executed.

4.1 A global proof rule

Let D denote differentiation and define

$$L_\alpha = D - \alpha, \quad L_\alpha f = f' - \alpha f.$$

Lemma 4.1 (One integrating-factor step). *Let f be continuously differentiable on $[0, \infty)$, let $\alpha \in \mathbb{R}$, and suppose $f(0) \geq 0$ and $L_\alpha f(x) \geq 0$ for all $x \geq 0$. Then $f(x) \geq 0$ for all $x \geq 0$.*

Proof. For $h(x) = e^{-\alpha x} f(x)$ the product rule gives

$$h'(x) = e^{-\alpha x} (f'(x) - \alpha f(x)) \geq 0.$$

Thus $h(x) \geq h(0) = f(0) \geq 0$. Multiply by the positive number $e^{\alpha x}$. Alternatively, apply the fundamental theorem of calculus on each finite interval $[0, x]$; no compactness assertion about the entire half-line is needed. $\square$

An *integrating-factor ladder* is a finite sequence

$$f_0 = f, \quad f_{i+1} = L_{\alpha_i} f_i \quad (0 \leq i < r),$$

with $f_i(0) \geq 0$ and an independently checkable proof of $f_r \geq 0$ on the half-line. Applying the lemma backwards proves the original goal.

4.2 An exact finite language

The implemented language is the rational exponential-polynomial space

$$f(x) = \sum_{\lambda \in \Lambda} p_\lambda(x) e^{\lambda x}, \quad \Lambda \subset \mathbb{Q} \text{ finite}, \quad p_\lambda \in \mathbb{Q}[x].$$

The transformation is exact and remains in the language:

$$L_\alpha(p_\lambda e^{\lambda x}) = (p'_\lambda + (\lambda - \alpha)p_\lambda) e^{\lambda x}. \quad (8)$$

A cheap terminal cone requires every coefficient of every p_λ to be nonnegative. Then each polynomial is nonnegative for $x \geq 0$ and every exponential is positive. This terminal cone is intentionally narrow; Forge’s existing polynomial worker can later replace it with stronger certificates.

Theorem 4.2 (Ladder certificate soundness). *Suppose the certificate checker verifies every identity (8), every boundary seed, and membership of f_r in the terminal cone. Then $f_0(x) \geq 0$ for all $x \geq 0$.*

Proof. Every exponential polynomial is continuously differentiable. The terminal coefficient test proves $f_r \geq 0$. Apply Lemma 4.1 for $i = r - 1, r - 2, \dots, 0$ using the corresponding seed and identity. $\square$

Example 4.3 (All orders of the exponential remainder). For an integer $s \geq 1$, set

$$f_s(x) = e^x - \sum_{j=0}^{s-1} \frac{x^j}{j!}.$$

One exact transformation gives

$$(D - 1)f_s(x) = \frac{x^{s-1}}{(s-1)!}.$$

Since $f_s(0) = 0$, a single ladder step proves $f_s(x) \geq 0$ for every $x \geq 0$. The complexity of this certificate does not depend on the width of a numerical box. For $s = 2$, the stored certificate uses rate 1 and terminal polynomial x .

4.3 Bounded search with a small candidate set

The search uses breadth-first exploration, offering rates in $\Lambda \cup \{0\}$. Each state is an exact canonical exponential polynomial. A state with a negative seed is discarded from this particular ladder search, not declared to make the original goal false. Canonical states are memoised; with unit-cost transitions, first discovery by breadth-first search has maximal remaining depth among paths to that state.

```

queue := [(source, empty_rate_list)]
while queue nonempty and node_budget remains:
    f, path := pop_front(queue)
    if coefficients of f are all nonnegative:
        return the exact stages, rates, and boundary seeds
    if depth budget exhausted: continue
    for alpha in source_rates union {0}:
        g := exact_derivative(f) - alpha * f
        if g(0) >= 0 and g is new:
            push_back(queue, (g, path + [alpha]))
return unknown

```

For s rate blocks of maximum degree d , one transformation uses $O(sd)$ rational arithmetic operations. The naive search tree has branching factor at most $s + 1$ and can be exponential in the depth cap. The experiments use a depth cap of 12 and a node cap of 4,000. The certificate checker accepts any admitted rational rates, not only the heuristic candidate set used by search. Therefore improving rate selection does not require enlarging the checker.

A concrete next rate heuristic is to inspect the leading coefficient in each block. Choosing $\alpha = \lambda$ removes the degree-preserving term from that block, leaving a derivative and reducing its degree. Candidate rates outside Λ could also be proposed by solving the linear inequalities in $\lambda - \alpha$ required by a terminal coefficient test. That extension is not implemented here.

4.4 A precise incompleteness obstruction

Consider the globally nonnegative function

$$F(x) = (e^x - 2)^2 = e^{2x} - 4e^x + 4.$$

At $a = \log 2 > 0$, $F(a) = F'(a) = 0$ and $F''(a) > 0$. For any finite real α , let $G = F' - \alpha F$. Then $G(a) = 0$ and $G'(a) = F''(a) > 0$. Hence $G(x) < 0$ immediately to the left of a . No first integrating-factor step can have a globally nonnegative residual. A zero-step certificate also fails the prototype's coefficient cone because the e^x coefficient is -4 .

Thus no choice of a longer ladder can repair this particular failure within the specified proof rule: the first required implication is already obstructed. The correct response is to expose the square and use the existing algebraic positivity machinery. The recorded depth-eight search returns *unknown*, as it should. This example is also a test against an implementation that checks only the seeds but forgets that every residual must ultimately be proved nonnegative on the whole domain.

4.5 What is and is not new

The mathematical rule is ordinary integrating-factor comparison. Mathlib already provides derivative-based comparison infrastructure [16]. The proposed capability is automatic *discovery* of a finite chain in an exact language, followed by a small compositional certificate. It is not a new differential-equation solver, a general transcendental decision procedure, or a claim that exponential-polynomial positivity is decided by this search.

5 Worker C: rational tail barriers

Evidence status. Constructive synthesis, coefficient optimisation, Python replay, finite-partial-sum checks, and modulus witnesses are implemented. The infinite semantic bridge is proved on paper and remains to be formalised in Lean.

5.1 Why a finite telescoper is not enough

An infinite-series theorem has obligations absent from a finite sum: the terms must be defined on the tail, finite initial exceptions must be separated, convergence must be established, and only then may the infinite-sum value be used. The prototype's subject therefore says *eventual rational tail*. It does not claim to interpret an arbitrary source function at poles of a rational expression.

Let a real sequence satisfy

$$a_n = \frac{A(n)}{D(n)} \quad (n \geq N), \quad (9)$$

where $A, D \in \mathbb{Q}[n]$, and let

$$B(n) = \frac{C}{(n+c)^p}, \quad C > 0, \quad p \in \mathbb{N}, \quad p \geq 1. \quad (10)$$

The source bridge must prove (9), including the interpretation of the natural-number index as a real number. The certificate will prove $D(n) > 0$ and $n+c \geq 1$ on the admitted tail.

Definition 5.1 (Tail barrier). A tail barrier at N consists of $a_n \geq 0$, $B(n) \geq 0$, and

$$a_n + B(n+1) \leq B(n) \quad (n \geq N). \quad (11)$$

Theorem 5.2 (Finite and infinite consequences). *Under these assumptions, for $M \geq N$,*

$$0 \leq \sum_{n=N}^M a_n \leq B(N) - B(M+1) \leq B(N). \quad (12)$$

The nonnegative tail series is summable, and

$$0 \leq \sum_{n=N}^{\infty} a_n \leq B(N).$$

The same statements hold with any larger starting index in place of N .

Proof. Sum (11) over the finite range and cancel the adjacent B terms. Nonnegativity of the a_n gives the left inequality; nonnegativity of $B(M+1)$ gives the last. Thus the tail partial sums form a nondecreasing bounded sequence of real numbers and converge. Taking the limit in the finite bound gives the tail estimate. Applying the same argument starting at a larger index proves the final statement. $\square$

A stronger bound on an individual finite range does not depend on any infinite-sum convention. In Lean, it should be the first lemma proved and tested. Summability is a separate theorem derived from it. Existing Mathlib infinite-sum and sum–integral comparison APIs are appropriate reusable infrastructure, not functionality to implement from scratch [17, 18].

5.2 A polynomial residual with all signs accounted for

Set $u(n) = (n+c)^p$ and $v(n) = (n+c+1)^p$. On the admitted tail, all of D, u, v are positive. Multiplication by Duv therefore preserves inequalities, and (11) is equivalent to

$$R(n) = C(v(n) - u(n))D(n) - A(n)u(n)v(n) \geq 0. \quad (13)$$

This orientation is easy to reverse accidentally: the correct telescoping quantity is $B(n) - B(n+1)$.

The checker uses the shift $n = N + t$. It requires all coefficients of $A(N+t)$, $D(N+t)$, and $R(N+t)$ to be nonnegative, with the constant coefficient of $D(N+t)$ strictly positive. Then for $t \geq 0$, numerator and residual are nonnegative and the denominator is strictly positive. It also checks $N+c \geq 1$. These finite rational checks imply every side condition in Theorem 5.2.

Example 5.3 (An exact telescoper). For $a_n = 1/(n(n+1))$, choose $N = 1$, $C = 1$, $c = 0$, and $p = 1$. Then $R = 0$ identically, so the checker recognises an exact telescoping identity as a special case of a barrier. It does not need a different infinite-series rule.

Example 5.4 (A searched inequality, not an exact telescoper). For $a_n = 1/n^2$ and $N = 2$, the coefficient optimiser chooses

$$B(n) = \frac{3}{2n}, \quad \sum_{n=2}^{\infty} \frac{1}{n^2} \leq \frac{3}{4}.$$

Here $R(n) = \frac{1}{2}n^2 - n$, so $R(2+t) = t + \frac{1}{2}t^2$ has nonnegative coefficients. This is an independently checkable rational inequality; a decimal approximation to the sum is unnecessary.

5.3 Exact optimisation of the barrier constant

For fixed N, c, p , write

$$R(N+t) = CU(t) - V(t).$$

For each coefficient, the requirement $CU_i - V_i \geq 0$ gives a half-line or an interval constraint on C . If $U_i > 0$, require $C \geq V_i/U_i$; if $U_i < 0$, require $C \leq V_i/U_i$; if $U_i = 0$, reject the template when $V_i > 0$. Intersect all constraints with $C > 0$. This solves the coefficient feasibility problem exactly with rational comparisons and does not need a linear-programming package.

The implementation enumerates a finite set of shifts and exponents, solves this one-dimensional feasibility problem for each, and keeps the smallest certified $B(N)$. Its default shift set is $\{-1, 0, 1\}$ and its exponent cap is eight. It is complete for each fixed template's *coefficientwise criterion*, not for all valid rational barriers or all nonnegative polynomials. A polynomial can be positive on a half-line despite having a negative shifted coefficient. A stronger existing Forge polynomial certificate can replace this criterion without changing the infinite-sum theorem.

If the feasible interval has infimum zero but no positive minimum, the implementation chooses a positive feasible value rather than asserting that an unattained optimum was attained. For a nonzero nonnegative tail, positive lower constraints normally force a positive minimum; the boundary convention still needs to be explicit.

6 A constructive classification for rational series

A bounded template search is useful, but this fragment permits more. The next result is a *certificate construction*: it explains why increasing the cutoff eventually succeeds and gives a computable cutoff, rather than relying on repeated trial and error.

6.1 Making every shifted coefficient positive

Lemma 6.1 (Positive-shift lemma). *Let $q(n) = \sum_{j=0}^m q_j n^j \in \mathbb{Q}[n]$ be nonzero with $q_m > 0$. Define, for $m > 0$,*

$$H = \max_{0 \leq k \leq j < m} \frac{\binom{j}{k} q_j}{\binom{m}{k} q_m},$$

and set $H = 0$ for $m = 0$. Then every coefficient of $q(N+t)$ is strictly positive whenever $N \geq \lfloor H \rfloor + 3$.

Proof. The coefficient of t^k is

$$g_k(N) = \sum_{j=k}^m \binom{j}{k} q_j N^{j-k}.$$

For $k = m$ this is $q_m > 0$. For $k < m$, put $h = \binom{m}{k} q_m > 0$ and $d = m - k \geq 1$. By the definition of H ,

$$g_k(N) \geq h \left(N^d - H \sum_{i=0}^{d-1} N^i \right).$$

Since $N > H + 1$ and $N > 1$,

$$H \sum_{i=0}^{d-1} N^i = H \frac{N^d - 1}{N - 1} < N^d.$$

Thus $g_k(N) > 0$. The constant-polynomial case is immediate. $\square$

The bound is conservative. The search may find a much smaller usable cutoff, but replay does not need to trust any root bound: it directly recomputes and checks the final shifted coefficients. The implementation tests this helper on 200 additional randomly generated positive-leading polynomials.

6.2 Convergence certificates

Theorem 6.2 (Constructive upper barrier). *Let $A, D \in \mathbb{Q}[n]$ be nonzero with positive leading coefficients a, d_0 , and let*

$$d = \deg D - \deg A \geq 2, \quad p = d - 1.$$

For every rational $C > a/(pd_0)$, there is an explicitly computable integer $N \geq 1$ for which the coefficient checker accepts $B(n) = C/n^p$ as a tail barrier for $A(n)/D(n)$.

Proof. With $c = 0$, the polynomial

$$R(n) = C((n+1)^p - n^p)D(n) - A(n)n^p(n+1)^p$$

has degree $\deg D + p - 1 = \deg A + 2p$. Its leading coefficient is $Cpd_0 - a > 0$. The polynomials A, D, R therefore all have positive leading coefficients. Apply Lemma 6.1 to each and choose N at least the maximum of the three resulting cutoffs and 1. Their shifted coefficients are strictly positive, hence all barrier checks hold. $\square$

The prototype chooses $C = a/(pd_0) + 1$. A smaller rational slack can improve the asymptotic constant at the expense of a potentially larger cutoff and larger coefficient sizes. This is a genuine tradeoff between the strength of the analytic result and the cost of its certificate.

6.3 A certified divergence outcome

Theorem 6.3 (Constructive harmonic minorant). *Under the same positive-leading assumptions, suppose $d = \deg D - \deg A \leq 1$. There are explicitly computable rational $c_0 > 0$ and integer $N \geq 1$ such that*

$$\frac{A(n)}{D(n)} \geq \frac{c_0}{n} \quad (n \geq N).$$

Consequently the positive tail partial sums are unbounded and the series is not summable.

Proof. If $d = 1$, choose $c_0 = a/(2d_0)$. Then $nA(n) - c_0D(n)$ has positive leading coefficient $a/2$. If $d \leq 0$, choose $c_0 = 1$; the term $nA(n)$ has degree strictly greater than $D(n)$ and positive leading coefficient. In either case apply Lemma 6.1 to D and $H(n) = nA(n) - c_0D(n)$. Beyond the resulting cutoff, $D(n) > 0$ and $H(n) \geq 0$. Division by the positive number $nD(n)$ yields the claimed minorant.

For an explicit divergence argument, group the harmonic tail into blocks $2^j N \leq n < 2^{j+1}N$. Each block contains $2^j N$ terms, each at least $1/(2^{j+1}N)$, so its sum is at least $1/2$. Every corresponding block of our series contributes at least $c_0/2$. Arbitrarily many blocks give unbounded partial sums. $\square$

The checker stores c_0 , N , the shifted denominator, and the shifted polynomial H . It recomputes these polynomials from the external source subject. The verdict means *divergence of the stated eventual rational tail under its bridge assumptions*. It does not mean that a failed barrier search has disproved convergence, and it does not misuse a real-valued `tsum` as though it were an extended-real infinity.

Corollary 6.4 (Unbounded-algorithm completeness on the stated fragment). *For nonzero rational A, D with positive leading coefficients, the construction produces a summability certificate when $\deg D - \deg A \geq 2$ and a divergence certificate when $\deg D - \deg A \leq 1$. Finite alterations of source values do not change the classification.*

Proof. Apply Theorems 6.2 and 6.3. A finite prefix has a finite real sum and cannot change convergence or unbounded tail growth. $\square$

The familiar convergence classification is not presented as new mathematics. The useful algorithmic result is the explicit compilation into one of two small certificate languages, using only rational polynomial arithmetic and a computed shift. The prototype has resource bounds, so the corollary is a statement about the uncapped construction, not a guarantee that every input is accepted by the shipped bounded checker.

The zero numerator is a trivial separate case and is excluded by the current constructor. Eventually negative rational sequences can be handled by a sign-normalisation adapter and absolute convergence, but that adapter is not implemented. Arbitrary oscillating, hypergeometric, or holonomic sequences are outside this classification. Forge's existing recurrence or telescoping workers may help prove a majorisation by a rational tail; the classification itself does not silently extend to those sequences.

6.4 From summability to asymptotic tail constants: a next extension

The same residual language suggests a small further development. Once an upper barrier has established summability, a lower potential satisfying

$$B_{\text{low}}(n) - B_{\text{low}}(n+1) \leq a_n$$

can be summed and passed to the limit. For the rational fragment, any positive constant below $a/(pd_0)$ eventually yields such a lower potential, by the same leading-coefficient argument with the residual sign reversed. Together with Theorem 6.2, this gives

$$\sum_{k=n}^{\infty} \frac{A(k)}{D(k)} \sim \frac{a}{pd_0} n^{-p}.$$

Here the argument is a mathematical extension specification, not an implemented second-polarity checker. The relevant point for Forge is concrete: a single extra residual polarity

could turn a convergence worker into a certified asymptotic-estimate worker without calling an unchecked symbolic limit engine. One must first establish summability with the upper barrier; reversing the residual does not by itself prove that an infinite sum exists.

7 Synthesising and checking Cauchy moduli

A summability theorem is often less useful to automation than an explicit answer to “how far must the finite approximation go?” A barrier supplies this answer without searching for a new induction invariant.

Let $\varepsilon \in \mathbb{Q}$ be positive and assume a validated barrier $C/(n+c)^p$ on $n \geq N$. A simple sufficient choice is

$$K = \max \left\{ N, 1 - c, \left\lfloor \frac{C}{\varepsilon} \right\rfloor + 1 - c \right\}. \quad (14)$$

Then $K + c > C/\varepsilon$ and $K + c \geq 1$, so $(K + c)^p \geq K + c > C/\varepsilon$. Therefore $B(K) < \varepsilon$. For every finite interval beginning at or beyond K , the sum of the nonnegative terms is less than ε . This is an executable Cauchy modulus for the tail partial sums; the index convention is “include all terms with index less than K in the approximation”.

7.1 The least admitted index

The simple bound can be wasteful when $p > 1$. Let

$$q = \lfloor C/\varepsilon \rfloor, \quad t = \lfloor q^{1/p} \rfloor + 1.$$

Because t^p is an integer, $t^p > C/\varepsilon$ is equivalent to $t^p > q$. Thus the least index within the certified tail is

$$K_* = \max\{N, t - c\}. \quad (15)$$

The implementation obtains t by integer doubling and bisection, with no floating-point root computation.

The witness is checked *without re-running the bisection*. Verify

$$K_* \geq N, \quad C < \varepsilon(K_* + c)^p,$$

and, unless $K_* = N$, also verify

$$C \geq \varepsilon(K_* - 1 + c)^p.$$

Strict monotonicity of the positive integer powers establishes leastness among indices at least N . These checks are exact rational inequalities. They do not ask the checker to trust how the candidate index was discovered.

In the accompanying experiment, 324 rational-accuracy requests were checked against the 81 barrier records. The least-index routine improved the coarse index in 115 requests. This measures witness quality inside the same certified barrier; it is not a claim that the underlying series has no better analytic tail estimate.

7.2 Rational accuracy versus an arbitrary real accuracy

The executable API accepts rational ε . For a Lean goal $\forall \varepsilon > 0, \exists K, \dots$ with real ε , one may use an Archimedean or rational-density theorem to obtain a positive rational accuracy smaller than it, then instantiate the rational witness theorem. Alternatively the proof may use a noncomputable real-floor expression. Neither route should be advertised as a numerical algorithm on an arbitrary opaque real value. This is an instance of Forge’s existing executable-versus-logical context distinction, applied to a new analytic witness [2].

8 How the workers cooperate with Forge

The proposal is not a new general controller. It adds three producers to the existing obligation mechanism, each with a precise applicability test and a specific output theorem. Existing source ownership and acceptance contracts remain prerequisites, not contributions to be rediscovered [6, 7].

8.1 Dispatch based on the shape of the obligation

Observed source shape	First action	What to export after checking
A compact analytic inequality with an exact zero at a rational anchor	Translate to the anchor and try a valuation-preserving model before flattening the error	A quantified bound $f(x) \geq \delta(x - a)^m$, including the anchor and the exact interval.
A global exponential-polynomial sign on a half-line	Try a bounded differential ladder, with existing polynomial positivity available for terminals	A global sign theorem, not a finite-box observation.
An infinite sum or requested convergence rate with eventual rational terms	Prove the eventual representation; construct a barrier or harmonic minorant	Summability and explicit tail bound, or a theorem of nonsummability; optionally a modulus witness.
An ordinary compact numerical bound with no delicate contact	Use LeanCert or another already verified enclosure library	The bound returned by the library under the chosen trust profile.
A residual is a perfect square or ordinary polynomial problem	Return to existing algebraic and polynomial workers	The polynomial theorem, avoiding unnecessary analytic approximation.

The prototype anchor is zero. Translation to a general rational anchor, two-sided intervals, and proofs involving symbolic parameters are controller extensions, not capabilities demonstrated by its AST. In particular a two-sided interval requires parity-aware handling of $(x - a)^m$; the positive-half-interval proof cannot simply be copied to an odd power on the other side.

8.2 A cross-worker example

Suppose the source context contains

$$0 \leq x \leq \frac{1}{2}, \quad x - \sin x \leq 0,$$

and the target is $g(x) = g(0)$ for an otherwise arbitrary function g . The anchored worker supplies (1); ordered arithmetic then derives $x = 0$, and congruence closes the target. The numerical worker needs no theory of g . The equality worker needs no Taylor series. The interface fact is the original-context inequality, not a new opaque Boolean result.

For a convergence proof, an existing recurrence or summation worker might establish $0 \leq u_n \leq A(n)/D(n)$ eventually. The new tail barrier supplies a summable majorant and an explicit tail estimate. Summability comparison then transfers the result to u . This is more useful than requiring the source sequence itself to be rational, but the comparison theorem and all index guards must be present. The prototype tests the rational source only; this transfer is a specified integration, not another executed corpus.

8.3 Failure diagnostics that change the next attempt

Each worker should distinguish a failed mathematical precondition from a resource limit. For anchored models, a failed logarithm guard should create a positivity/domain obligation or trigger a change of centre; a negative interval margin should suggest a higher order, a smaller interval, or a stronger polynomial quotient check. For ladders, a negative intermediate seed rejects a path, whereas the interior-double-zero obstruction suggests factorisation or splitting. For rational tails, a failed coefficient test at a requested cutoff should suggest moving the cutoff or using a stronger polynomial certificate.

These are routing suggestions, not accepted facts. Conversely, a successfully checked harmonic minorant is a mathematical divergence result in its named fragment. It must not be collapsed into the same status as “no upper barrier found.”

8.4 A concrete budget policy

Use separate budgets for source translation, candidate generation, and proof reconstruction, as the existing Forge architecture requires. Within the new lanes, order work by measurable quantities: expression-node count, polynomial degree, denominator bit length, number of rate blocks, residual degree, and estimated proof-term expansion. For example, the tail worker should reject an exponent exceeding the admitted checker limit *before* expanding $(n + c)^p$.

A practical initial routing policy is: try cheap source simplification first; run a bounded ladder for a recognised global exponential polynomial; use an anchored model for exact-contact compact goals; otherwise invoke the existing verified enclosure worker. Infinite-series goals enter the tail lane only after an eventual source equation is proved. This is a deterministic policy that can be evaluated against the same workers without communication; a learned scheduler is neither required nor tested here.

9 Libraries, version boundaries, and proof reconstruction

9.1 Reuse LeanCert; do not write a second interval library

The inspected LeanCert README describes certified interval evaluation, transcendental functions, automatic differentiation, root certificates, optimisation, finite sums, integrals, natural-number tail bounds, and checked unary enclosure extension rules. It exposes, among other names, `LeanCert.evalInterval`, `LeanCert.evalInterval_correct`, `LeanCert.API.Bounds`, and `LeanCert.API.Optimization` [11]. It also already uses Taylor-model machinery. Therefore this article does *not* claim that analytic enclosures, tails in every sense, or Taylor models are absent from the Lean ecosystem.

The recommended division of labour is specific. Use LeanCert’s checked primitive enclosures and ordinary box bounds. Let Forge select an exact-contact quotient, organise a global differential ladder, or request a series barrier. Where a library API already proves the exact desired anchored statement, use that theorem instead of the prototype’s weaker primitive majorant. A full capability comparison against LeanCert was not performed, and no exclusive capability claim relative to it is justified by these experiments.

The Python package is useful for independent numerical search and advertises a bundled Lean bridge on supported platforms. Installing it was attempted here, but network/DNS access in the execution container failed. The bridge was not run. The executable prototypes in this archive therefore do not depend on it.

9.2 The observed toolchain mismatch

Forge’s README pins Lean v4.34.0 and Mathlib revision

```
1cf325a0cf67aca2b04d76b5380ff6a9e410aefa
```

The inspected LeanCert `lean-toolchain` says v4.33.1, and its Lake dependency selects Mathlib v4.33.1 [1, 12]. These are not a tested single dependency set. This archive deliberately does not invent a Lake manifest asserting that they build together.

There are two honest integration options. Port a narrowly selected LeanCert adapter to Forge’s pin and compile it there, or develop the adapter against LeanCert’s pin while clearly postponing the Forge merge. An out-of-process worker can exchange rational *data* across those versions, but proof acceptance must occur in the target environment. A proof term from another version is not accepted merely because its printed theorem statement matches.

9.3 Two trust choices, and one concrete import hazard

LeanCert documents a default native checking mode using `native_decide`, a kernel mode using `decide +kernel` with no fallback, and an automatic mode that can report a native fallback. Forge must request the intended mode explicitly; a numerical backend choice such as rational versus dyadic is a different axis [11, 13].

A narrower source observation also matters. The inspected `LeanCert/CertifiedBounds/Li2.lean` intentionally states two numerical bounds with `sorry`; its documentation says matching statements are proved in a separate heavy CI target. The module is explicitly not a forwarding module [14]. This does not imply that LeanCert’s main checkers are unproved. It does imply that a Forge adapter must not treat every theorem under a library’s namespace as equally suitable proof authority.

Use the actual verified declaration, or reconstruct the needed result, rather than importing the lightweight placeholder theorem. Separately proving an equal statement does not replace the proof body of a declaration that depends on `sorryAx`. The appropriate audit follows the transitive dependencies of the *accepted theorem*; the mere presence of an unused placeholder elsewhere in the environment is not itself a dependency of that theorem. No Li2 module is needed for the algorithms proposed here.

9.4 A concrete dependency map

Component	Reuse	New work actually required
Lean core and existing Forge controller	Original goal identity, expression construction, local hypotheses, acceptance	Register three applicability predicates and their source-specific residual obligations.
Mathlib polynomial and tactic infrastructure	Exact polynomial identities, ordered arithmetic, powers, finite sums	A reflected shifted-coefficient positivity theorem and a small certificate decoder.
Mathlib calculus/Taylor modules	Derivative rules, Taylor theory, comparison infrastructure	Connect an admitted jet or differential ladder to the exact source function.

Component	Reuse	New work actually required
Mathlib infinite-sum infrastructure	Convergence of nonnegative bounded partial sums, finite-prefix transport, comparison	A generic barrier theorem, harmonic-minorant theorem, and index-safe instantiation.
LeanCert	Verified elementary enclosures, ordinary compact numerical bounds, optional AD	A version-pinned adapter preserving the anchor and selecting a declared trust mode.
Python <code>fractions</code> , integer arithmetic	Portable exact search and prototype replay	Already implemented; never substitute Python acceptance for the Lean bridge.
SymPy and mpmath	Independent symbolic diagnostics and high-precision numerical samples	Testing only. They do not appear in the shipped search/replay dependency chain.

Mathlib’s documented Taylor definitions and derivative-comparison lemmas are source-level integration targets, not proof scripts compiled in this session [15, 16]. Verify exact names and signatures under the selected pin before implementing an adapter. In particular, assumptions about differentiability within an interval and endpoint continuity must not be lost when selecting a convenient comparison theorem.

10 The missing Lean layer, specified narrowly

The easiest first implementation among these *new* lanes is the rational tail worker. It needs exact polynomial checking, finite telescoping, and a summability theorem, but no large catalogue of analytic primitive rules. This recommendation should not delay Forge’s already proposed narrow end-to-end milestone: the existing finite-algebra cover remains a reasonable first integration test [3]. The new lanes are modular extensions, not a reason to restart the project.

10.1 Proposed module boundaries

```
Forge/Analysis/Source.lean
Forge/Analysis/Anchor/Semantics.lean
Forge/Analysis/Anchor/Checker.lean
Forge/Analysis/Anchor/Soundness.lean
Forge/Analysis/Differential/Ladder.lean
Forge/Analysis/RationalTail/Shift.lean
Forge/Analysis/RationalTail/Checker.lean
Forge/Analysis/RationalTail/Soundness.lean
Forge/Analysis/RationalTail/Modulus.lean
Forge/Analysis/Adapters/LeanCert.lean
Forge/Analysis/Route.lean
```

These are proposed paths, not existing source names. The architecture keeps source translation distinct from checker soundness. A reflected polynomial check can be perfectly correct while proving something about the wrong source sequence or wrong cast.

10.2 The three theorem families to formalise

For an anchored enclosure, prove a generic theorem with the semantic premise (2) and the checked quotient bound. Separately prove every AST constructor preserves the enclosure. The final source theorem combines the reification equality, the model soundness theorem, and Theorem 3.4. No theorem may assume “the Python model is correct” as an axiom.

For ladders, first prove Lemma 4.1; then prove soundness of a list of admitted stages by induction on the list. The exact derivative identity for an exponential polynomial is reconstructed from the coefficient data using ordinary derivative rules and ring normalisation. The terminal polynomial signs can be proved by existing polynomial tools, rather than baking the current coefficient cone into the outer ladder theorem.

For tails, first prove (12) by induction on the finite range length. Next prove the shifted-coefficient sign lemma; next connect (13) to the barrier step using positive denominator guards. Then prove summability and the tail estimate from bounded monotone partial sums. Only after these steps should the tactic construct an infinite-sum equality or inequality in the source goal. The harmonic-minorant theorem is a separate soundness theorem with a separate conclusion.

10.3 Source obligations that must be explicit

For analytic models, retain the exact source variable, function occurrence, rational scale, anchor, interval, and side of the anchor. A symbolic scale is not a rational constant simply because a current numerical model has assigned it a value. A logarithm node requires its original positive-argument proof. Unsupported composition must return an obligation or remain opaque; it must not be normalised into a different admitted primitive.

For tails, retain the sequence and a proof of the eventual equation, the starting index, the denominator sign theorem, and the casts from naturals to reals. Natural subtraction is not field subtraction. A requested bound from index N_0 cannot be replaced by a bound from a larger discovered N without adding the intervening finite sum. The construction can prove convergence after moving the cutoff, but a quantitative requested tail bound has stricter indexing requirements.

For ladders, retain the exact real-valued source function and the half-line. Boundary values must be exact rational evaluations, not floating-point approximations. The same differential identity on an interval that misses zero does not license use of the seed at zero without a bridge over the gap.

These requirements are the analytic application of source-owned evidence already developed in Djex and Leant. Their source-typed graph design explicitly associates evidence with a particular candidate and checked inventory, rather than inferring authority from a displayed global name [7]. Nothing in this proposal resolves their separate open Church-indexing or universe-composition synthesis tasks.

10.4 What the supplied Lean file demonstrates

The archive contains `lean/SemanticBridges.lean`, with two small candidate lemmas: non-negativity from an already established anchored factorisation, and the finite telescoping consequence of an already established barrier step. They contain no admitted proofs. They are deliberately modest: neither lemma checks a JSON certificate or reifies a source expression. The file was not compiled here, and it must not be counted as a kernel-checked artifact.

Providing these candidates is useful for fixing the intended proposition and index convention.

Calling them an implemented tactic would not be useful. The substantive missing work is the checker-soundness/source-bridge composition, followed by actual kernel acceptance and axiom-dependency inspection.

11 Experimental evidence and its limits

11.1 Environment and reproduction

The recorded run used Python 3.13.5 on Linux x86-64 with glibc 2.41. The optional oracle run used SymPy 1.14.0 and mpmath 1.3.0 at 120 decimal digits. The random seed is 20260915. The exact commands, problem subjects, certificates, mutation outcomes, and unindented JSON byte measurements are retained. No Lean, `lake`, or `elan` executable was found. Attempts to install or clone through the execution container failed because external host names could not be resolved; GitHub source inspection itself was available through the connected read interface.

Certificate lane	Records	Distinct subjects	Largest JSON record
Anchored enclosures	62	61	339 bytes; order 8
Differential ladders	43	42	671 bytes; depth 4
Upper tail barriers	81	67	878 bytes; cutoff 21
Harmonic minorants	60	60	387 bytes; cutoff 12
All certificate records	246	230	Not a Lean proof-size metric

The totals in this table are counts of the same object, namely certificate records, so adding them is meaningful. They are not added to unit tests, mutations, numeric samples, or finite-sum checks. Nor are they a solved-goal benchmark. The repeated subjects are disclosed in `results/duplicate-audit.json`.

11.2 Anchored corpus and ablation

The corpus consists of 48 scale/radius combinations of four elementary gaps, eight products of two such gaps, and six higher-order exponential remainders. The elementary families are

$$e^{ax} - 1 - ax, \quad ax - \sin(ax), \quad ax - \log(1 + ax), \quad ax - \arctan(ax),$$

with $a \in \{1/4, 1/2, 1, 2\}$ and $ab \in \{1/4, 1/2, 3/4\}$. The higher-order subfamily repeats one subject from the elementary family.

All 62 records are accepted. The factor-erasing ablation uses the *same* expression, box, chosen order, and remainder model, but applies the lower bound from $H_b(p) + [0, b^K]I$ without valuation extraction. It succeeds in 8 of the 62 records. Those eight are the product subfamily: the model already has zero truncated polynomial and a positive remainder interval, so the simple bound suffices.

On distinct subjects the figures are 61 accepted by anchored extraction versus 8 accepted by the ablation. This isolates a representation loss, not a comparison against a well-tuned interval prover. Signed remainder rules, symbolic factorisation, a stronger Taylor implementation, or an existing library theorem can also solve some or all of these examples. In particular, the product examples have intentionally simple mathematical structure. The ablation is not evidence that the new worker outperforms LeanCert.

SymPy independently reconstructs all 62 Taylor polynomials from the function expressions. A separate mpmath evaluator checks 620 nonzero sample points against the stored remainder intervals. Sampling is diagnostic: it cannot certify a universal interval claim and is not used by the acceptance checker.

11.3 Ladder corpus and its planted component

There are seven hand-described exponential-remainder records, including one repeated subject, and 36 planted examples. A planted example begins with a terminal-cone exponential polynomial g and repeatedly solves

$$(D - \alpha)f = g, \quad f(0) = c \geq 0$$

exactly within the exponential-polynomial language. This deliberately creates inputs with known ladder certificates. The search is then run from the resulting source function, without being given the planted rate list.

All 43 records are accepted. The checker validates the displayed stages, not the construction history. SymPy cross-checks 68 differential identities in the returned ladders. Planted examples test search/replay consistency and sensitivity to degree/rate structure. They are not representative samples from arbitrary user inequalities. The always-nonnegative function $(e^x - 2)^2$ is retained as an expected **unknown** control, and its obstruction is proved in Section 4 rather than explained away as insufficient fuel.

11.4 Tail corpus, witnesses, and exact controls

The rational-function generator creates 120 subjects with random coefficients and positive leading coefficients. Numerator degrees lie between zero and four; requested degree offsets cycle through $-2, -1, 0, 1, 2, 3, 4, 5$, with denominator degree clamped below by zero. The construction produces 60 upper barriers and 60 harmonic minorants. Unlike the ladder corpus, these rational subjects are not obtained by inverting the certificate equations; the completeness theorem explains why the appropriate certificate exists.

For the 60 constructed upper barriers, 4,800 exact finite partial-sum inequalities are checked, and 240 coarse rational-accuracy witnesses are checked. For the 60 divergence records, 2,400 exact pointwise harmonic comparisons are checked. These finite checks are sanity checks of the infinite certificate, not a substitute for its mathematical proof.

An additional 21 barrier records optimise $1/n^s$ for $s = 2, \dots, 8$ at cutoffs 2, 5, 10. They concern seven sequences and three quantitative requests per sequence. The optional SymPy run verifies the shifted residual/denominator identities in all 141 tail records. Finally 324 least-index witnesses, four for each of the 81 upper barrier records, pass their exact two-inequality checks; 115 improve the coarse witness.

11.5 Replay, mutations, and explicit non-successes

The standalone replay is invoked with Python's `-S` option, disabling site packages. Its import guard rejects attempts to import the search modules or numerical/symbolic libraries. All 246 records replay. This establishes a separable implementation path, not a formally verified checker.

Three evidence-bearing mutations are applied to every record: source subject alteration, a bound/seed/minorant alteration, and a polynomial or stage alteration. All 738 are rejected. Additional unit tests reject duplicate JSON keys, floating-point JSON, noncanonical rational

strings, Boolean values masquerading as integer cutoffs, and a denominator vanishing at the requested cutoff. The standard-library test suite contains 25 test methods; all pass.

The recorded analytic controls include four false elementary inequalities, an unmet logarithm-series domain, and the true trigonometric identity discussed earlier. All return `unknown` under the stated caps. The global even-root square is also `unknown`. These outcomes are not packaged as refutations. Only a separately validated harmonic minorant authorises the divergence conclusion in the rational-series lane.

11.6 What remains unmeasured

There is no compiled Lean checker, source reifier, Forge tactic call, or LeanCert adapter in this experiment. There is no Lean axiom audit, no kernel replay time, no proof-term size measurement, and no comparison against `grind`, `nlinarith`, a tuned LeanCert configuration, or a noncommunicating portfolio. The small JSON certificates suggest a manageable transport format; they say nothing by themselves about the size or cost of the resulting Lean proof terms.

The Python checker and search use different representations but share the same mathematical design. A common error in the derivation could survive both. The paper proofs, symbolic oracles, numeric diagnostics, and mutations attack different failure modes; none makes the checker formally verified. Likewise, input caps are basic resource guards, not a security audit against adversarial resource exhaustion.

12 An implementation and evaluation programme

12.1 Acceptance gates rather than speculative deadlines

The first gate is the finite barrier lemma and a polynomial sign checker in Lean under one chosen toolchain. Acceptance requires a source theorem about $\sum_{i=0}^{m-1} a(N+i)$, not merely a coefficient identity, and a recorded axiom inventory. Wrong-denominator and off-by-one variants must be rejected.

The second gate adds the generic infinite-tail theorem and finite-prefix transport. Acceptance requires both a positive convergence example and a harmonic-divergence example, with the exact original source sequence preserved. A passing finite-sum lemma alone does not close this gate.

The third gate adds coefficient-driven synthesis and least-index witnesses. Acceptance requires replay with the search disabled, including verification of the strict inequality at the chosen index and the predecessor inequality when leastness is claimed. The convergence theorem must continue to compile after removing Python and all optional search dependencies.

The fourth gate proves and instantiates the integrating-factor step. A global exponential remainder should compile from a generated ladder, while a fabricated ladder for the even-root square must fail. A proof over a finite box is not an acceptable substitute for the original half-line goal.

The fifth gate connects anchored models to source functions, reusing checked primitive enclosures where possible. Acceptance requires an endpoint-contact example, a product with high-order cancellation, a failed logarithm guard, and an exact identity routed away from interval approximation. Only then should these workers enter the normal Forge scheduler.

12.2 The benchmark that would demonstrate practical value

Evaluate four configurations on the same held-out source goals and identical premise sets: existing Forge without the new lanes; the same new workers as a noncommunicating portfolio; the communicating integration; and strong directly invoked alternatives including tuned LeanCert for supported numeric goals. Fix compiler versions, imports, theorem-retrieval restrictions, and budgets. Remove any target theorem or trivially equivalent generated lemma from retrieval.

Stratify the analytic goals by anchor order and number of cancelling terms; stratify global inequalities by rate blocks and terminal polynomial difficulty; stratify tails by rational degree gap, mixed coefficients, requested cutoff, and accuracy. Include false, unsupported, and deliberately difficult true controls. Report successful kernel proofs, unknowns, incorrect acceptances, source-translation failures, search time, replay time, proof size, and imported axioms separately.

The main hypothesis is not that every new worker is stronger than every existing tactic. It is that preserving contact order, finding a global comparison chain, and generating quantitative tail witnesses will close source obligations that currently require manual mathematical restructuring. That hypothesis becomes credible only when tested on actual Lean goals, with the source and proof boundary included in the measurement.

13 Conclusion

Three additions can broaden Forge without rebuilding its existing foundations. Anchored remainders retain the local order of vanishing that a uniform numerical error can erase. Integrating-factor ladders turn a global analytic sign into a finite chain of exact differential identities. Rational tail barriers turn infinite convergence into shifted polynomial signs, with explicit divergence certificates on the complementary eventually positive rational fragment and checked quantitative witnesses when convergence holds.

The constructions are concrete enough to implement and test: exact ASTs, coefficient formulas, finite certificate formats, bounded search policies, checker rules, soundness arguments, and a constructive completeness theorem are all supplied. Their limitations are equally concrete: a narrow analytic grammar, loss of correlations, a first-order global-comparison obstruction, rational-only tail classification, and an unimplemented Lean source/checker bridge.

The recommended contribution to the repository is therefore a small new **Analysis** extension area with the supplied prototypes and a gate-driven formalisation plan, not a replacement for **grind** and not another general trust-system redesign. The next decisive result is a kernel-checked source theorem through one of these lanes. The evidence supplied here makes that implementation specific, while leaving its unfinished status visible.

A Certificate schemas and replay contract

Rationals are canonical strings "**numerator/denominator**", with positive denominator, reduced form, and no leading plus or redundant zeros. Polynomial coefficients are in increasing degree order; the zero polynomial is ["0/1"]. The search uses dense tuples, while replay converts them to sparse maps with zero coefficients removed. Exponential blocks are sorted by rational rate and must not repeat a rate.

Every certificate carries a subject for transport, but the verifier also takes the expected subject *externally*. It first requires exact agreement. In a Lean integration that external

subject must be derived from the original goal and its proved source translation, not read back from the candidate.

```

anchored:
  kind, subject={expr,b}, order, polynomial,
  error=[lower,upper], valuation, margin

ladder:
  kind, subject={exp_poly,domain="x>=0"},
  rates, stages, seeds, search_nodes

barrier:
  kind, subject={numerator,denominator,
                interpretation="eventual-rational-tail"},
  cutoff, shift, power, constant, shifted_numerator,
  shifted_denominator, shifted_residual, tail_bound

divergence:
  kind, subject={numerator,denominator,
                interpretation="eventual-rational-tail"},
  cutoff, minorant, shifted_denominator, shifted_residual

```

The ladder's `search_nodes` is descriptive metadata. It is bounded by the decoder but is not evidence of correctness or completeness. Every mathematical field is checked or recomputed. Unexpected fields are rejected by the current version, so adding a new certificate rule requires an explicit schema change.

Principal replay limits are a 2 MB JSON envelope, 400 characters per rational, 1,024 bits in each admitted rational numerator/denominator, polynomial input length at most 193, anchored order at most 40, at most 32 exponential blocks and 32 ladder steps, tail exponent at most 16, and tail cutoff at most 10^6 . These caps bound admitted inputs, not the bit lengths of every intermediate arithmetic result. A production checker should add independently accounted intermediate-allocation and proof-reconstruction limits.

B Using the archive

From the archive root, no additional Python package is needed for these commands:

```

python -m unittest discover -s tests -v
python -S tools/verify.py
python tools/experiments.py --output reproduced-results
python tools/modulus_checks.py --output reproduced-moduli.json

```

The experiment runner refuses an existing output directory, preventing a fresh run from silently replacing the recorded evidence. It writes a failure receipt if an assertion fails. The optional oracle diagnostics require the versions in `requirements-testing.txt`:

```

python -m pip install -r requirements-testing.txt
python tools/oracle_checks.py --output reproduced-oracle.json

```

A minimal programmatic use is:

```
from fractions import Fraction as Q
from forge_analytic import jets as J
from forge_analytic.checker import verify

expr = J.minus(J.P(0, 1), J.atom('sin'))
expected = {'expr': expr, 'b': '1/2'}
candidate = J.search(expr, Q(1, 2))
if candidate is not None and verify(expected, candidate):
    print(candidate['valuation'], candidate['margin'])
# 3 31/216
# This is Python evidence, not a Lean proof.
```

Set `PYTHONPATH=prototype` or insert that directory in the Python module path before using the package directly. The command-line tools do this internally.

To rebuild the article:

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error forge_analytic_extensions.tex
pdflatex -interaction=nonstopmode -halt-on-error forge_analytic_extensions.tex
pdflatex -interaction=nonstopmode -halt-on-error forge_analytic_extensions.tex
```

The document uses ordinary pdfL^AT_EX packages, no shell escape, no external bibliography processor, and no redistributed font files.

C Source audit and attribution

The source inventory in `results/source-audit.json` records the repository revision and paths inspected. The audit is scoped to the items discussed in Section 1; it is not an exhaustive textual similarity audit of all frozen proposals. The numerical literature is credited for the underlying techniques. The soundness arguments and specialisations in this article are written out so the reader can assess the exact admitted fragment without relying on an informal appeal to a solver.

The code in this archive is original prototype code for these extensions. No third-party repository archive, package binary, or font is redistributed. Referenced projects and papers retain their own licences. The accompanying `STATUS.md` distinguishes executed Python artifacts, optional oracle checks, uncompiled Lean candidates, and unimplemented integration work.

References

- [1] V. Reshetnikov, *Forge*, README at audited revision `c98e47c5` (full identifier in the source audit), inspected 15 September 2026. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/README.md>.
- [2] Forge, *Provenance: what came from where*, merged article appendix B, same revision. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/B-provenance.tex>.
- [3] Forge, *Assessment and recommended starting point*, merged article section 12, same revision. <https://github.com/VladimirReshetnikov/Forge/blob/>

- c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/12-conclusion.tex.
- [4] Forge, *A strong baseline, not a straw man*, merged article section 2, same revision. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/02-baseline.tex>.
 - [5] Forge, merged article section 7, nonlinear certificates, especially the univariate, product-envelope, and domain discussion, same revision. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/07-nonlinear.tex>.
 - [6] Forge, *Ideas from Leant and Djex*, same revision. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/docs/IDEAS-FROM-LEANT-DJEX.md>.
 - [7] V. Reshetnikov, Djex, *Djinn source-typed evidence graph*, inspected 15 September 2026; source blob 7c1dca61d49f6e113c44fb9aba73d3203296cd5d. <https://github.com/VladimirReshetnikov/Djex/blob/main/docs/source-typed-evidence-graph.md>.
 - [8] V. Reshetnikov, *Leant—a Djex-based synthesis REPL for Lean 4*, README, inspected 15 September 2026. <https://github.com/VladimirReshetnikov/Leant>.
 - [9] M. Streeter and J. V. Dillon, *Automatically Bounding the Taylor Remainder Series: Tighter Bounds and New Applications*, arXiv:2212.11429, version 3, 2 August 2023. <https://arxiv.org/abs/2212.11429v3>.
 - [10] M. Daumas, D. Lester, and C. Muñoz, *Verified Real Number Calculations: A Library for Interval Arithmetic*, arXiv:0708.3721, 2007. <https://arxiv.org/abs/0708.3721>.
 - [11] LeanCert contributors, *LeanCert: Certified numerics for Lean 4*, README, inspected 15 September 2026; source blob 8a65c45afc1763a36d5d406692779daff401b92f. <https://github.com/alerad/leancert/blob/main/README.md>.
 - [12] LeanCert contributors, *lean-toolchain* and *lakefile.toml*, inspected 15 September 2026; Lean and Mathlib pins v4.33.1. <https://github.com/alerad/leancert/blob/main/lean-toolchain>; <https://github.com/alerad/leancert/blob/main/lakefile.toml>.
 - [13] LeanCert, *Trust model*, official documentation, inspected 15 September 2026. <https://docs.leancert.io/architecture/trust-model/>.
 - [14] LeanCert contributors, *LeanCert/CertifiedBounds/Li2.lean*, inspected 15 September 2026; source blob 5833f2515cbb42d1a2bd21fcdaedc5362be7cbaf. <https://github.com/alerad/leancert/blob/main/LeanCert/CertifiedBounds/Li2.lean>.
 - [15] Mathlib contributors, *Mathlib.Analysis.Calculus.Taylor*, official API reference, inspected 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Analysis/Calculus/Taylor.html.
 - [16] Mathlib contributors, *Mathlib.Analysis.Calculus.MeanValue*, official API reference, especially derivative comparison on finite intervals, inspected 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Analysis/Calculus/MeanValue.html.

- [17] Mathlib contributors, `Mathlib.Topology.Algebra.InfiniteSum.Order`, official API reference, inspected 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Topology/Algebra/InfiniteSum/Order.html.
- [18] Mathlib contributors, `Mathlib.Analysis.SumIntegralComparisons`, official API reference, inspected 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Analysis/SumIntegralComparisons.html.
- [19] Lean language reference, *The grind tactic*, current official reference, inspected 15 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.